ElemOS: An Elementary Kernel

Project Proposal — CSE323: Operating Systems

Name: Kazi Sujat Ahmed Sazid **Student ID:** 2231177642
Department of Computer Science & Engineering April 2, 2026

Project Overview

ElemOS is a minimal, single-address-space kernel written in C and x86-64 assembly, targeting the QEMU virtual machine. The goal is to build a working OS core from scratch — from the boot sector to a basic shell — demonstrating the fundamental mechanisms studied in CSE323: process scheduling, memory management, interrupt handling, and inter-process communication.

Objectives

- Boot a bare-metal system using a custom GRUB-compatible bootloader (Multiboot2).
- Implement a **Global Descriptor Table (GDT)** and **Interrupt Descriptor Table (IDT)** with a working ISR/IRQ pipeline.
- Build a **physical memory manager** (bitmap allocator) and a basic **virtual memory** layer using x86 paging.
- Develop a **round-robin process scheduler** supporting cooperative multitasking with context switching.
- Provide a rudimentary VGA/serial **console driver** and a minimal interactive shell.
- (Stretch) Add a simple **FAT12 filesystem** driver for a virtual floppy image.

Design & Architecture

ElemOS will follow a monolithic kernel design, partitioned into clearly separated modules:

Module	Responsibility
boot/	Multiboot2 entry, stack setup, calling `kernel_main()`
arch/x86/	GDT, IDT, ISR stubs, PIC remapping, CPUID
mm/	Physical frame allocator, page-table management, `kmalloc()`
proc/	PCB structure, scheduler, context switch (`switch_to()`)
drivers/	VGA framebuffer, 8250 UART, PS/2 keyboard
fs/ (stretch)	FAT12 read-only driver
lib/	`kprintf()`, string utilities, panic handler

Timeline

Week	Milestone	Deliverable
Week 1	Bootloader + GDT/IDT	Kernel boots; interrupts handled
Week 2	Memory management	`kmalloc` working; paging enabled
Week 3	Process & scheduler + Drivers	Tasks switching; console operational
Week 4	Shell, Testing & Report	Final demo, written report, clean repo

Tools & Environment

Language: C (C11) and x86-64 AT&T assembly **Toolchain:** GCC cross-compiler (`x86_64-elf`), GNU Make, NASM
Emulator: QEMU (`qemu-system-x86_64`) **Debugger:** GDB with QEMU remote stub
Version Control: Git (GitHub private repo)

Expected Outcomes

By the end of the project, ElemOS will successfully **boot in QEMU**, handle keyboard interrupts, schedule two or more concurrent processes, and respond to simple shell commands (`help`, `mem`, `ps`, `clear`). The submission will include full source code with documentation, a build guide, and a written report correlating implementation decisions to concepts from CSE323 lectures.

All work will be individual unless otherwise approved. Academic integrity policies apply.